

第6章 面向对象

建议学时:4-6 小时 | 难度:★★★★☆ | 读者背景:已掌握 C 语言

🎯 本章学习目标

- 理解从 C 的 struct + 函数指针到类的演进,以及"面向对象是组织代码的方式"
- 掌握原型链机制:prototype、__proto__、Object.create 三者关系
- 彻底理解 class 只是语法糖:背后仍是构造函数与原型链
- 掌握继承 extends 与 super,理解方法的查找链
- 掌握访问控制:JS 的 # 私有字段与私有方法
- 掌握 getter/setter、静态成员与 instanceof
- 理解多态与鸭子类型,以及组合优于继承的设计原则

💡 从 C 到 JavaScript:本章主题如何迁移

C 语言里,数据与行为是分开的:struct 只存数据,函数操作结构体靠"传指针"。想模拟多态,得手动维护函数指针表: `struct Shape { void (*area)(const void*); }` ——把"类型行为"硬编码进数据里,还要自己处理回调里的强制转换,笨拙且危险。

JavaScript 提供了两条路:传统原型链(ES5 时代)与现代 class 语法(ES2015+)。class 只是语法糖,底层依然是构造函数 + 原型链。理解原型链,你才能解释: `arr.push` 为什么存在、`instanceof` 怎么工作、猴子补丁为什么危险、框架为什么能改写原型方法。

与 C++/Java 不同,JS 的继承是 **基于原型的单链继承**:每个对象有一个原型(proto),属性查找沿着链向上走。没有多继承、没有虚函数表、没有访问修饰符的 `public/protected`——私有用 `#` 前缀实现(ES2022),访问控制在语言层面只有"公开/私有"两档。

C 的程序员最容易犯的错是把 JS 当 C++ 用:建一堆层级很深的继承树。JS 社区的主流实践是 **组合优先**:小对象组合成大对象、mixin 混入、鸭子类型。这不是规范强制的,而是生产经验——原型继承浅而组合宽,可维护性高得多。

📦 前置知识自查

- C 的 struct、指针、结构体数组
- C 的函数指针成员(struct + 函数指针模拟"方法")
- C 的静态库组织思想(头文件声明、实现分离)
- 第4章 this 四规则(本章的方法调用依赖隐式绑定)
- 第1章 typeof/对象基础

本章学习路线

1. **第一阶段(对象与原型)**:读 §6.1–§6.2,在 REPL 里检查原型链
2. **第二阶段(class 与继承)**:读 §6.3–§6.5,亲手写出"类→原型"的等价展开
3. **第三阶段(成员与设计)**:读 §6.6–§6.9,重点理解 # 私有与组合优于继承
4. **第四阶段(实验)**:完成章末"几何图形系统"
5. **验收标准**:不看资料,能用纯函数 + Object.create 手工实现一个"类"并讲清查找链

本章知识导图

- 第6章 面向对象
 - §6.1 从 C 到面向对象:struct+函数指针的演进
 - §6.2 对象与原型链:prototype / __proto__ / Object.create
 - §6.3 构造函数与 new 的过程
 - §6.4 class 语法糖:本质是原型,extends/super
 - §6.5 访问控制:# 私有字段与私有方法
 - §6.6 getter/setter 与访问器属性
 - §6.7 静态成员与 instanceof
 - §6.8 多态与鸭子类型
 - §6.9 设计原则:组合优于继承

§6.1 从 C 到面向对象:先看 C 的笨拙

```
// C 的 struct + 函数指针模拟"方法"(伪代码,展示痛点)
// typedef struct Shape {
//   double (*area)(const struct Shape* self); // 函数指针成员
//   double w, h;
// } Shape;
// 调用:shape.area(&shape) — "self" 要自己传,强转易错

// JS 的进化:先看"对象方法"的自然形态
const rect = {
  w: 4,
  h: 5,
  area() {
    return this.w * this.h; // this 自动是 rect,不用手动传
  },
};
console.log(rect.area()); // 20

// 工厂函数批量造对象(C 的结构体数组 → JS 的对象数组)
function makeRect(w, h) {
  return {
    w, h,
    area() { return this.w * this.h; },
  };
}
const shapes = [makeRect(4, 5), makeRect(2, 3)];
console.log(shapes[0].area()); // 20
console.log(shapes.map((s) => s.area()).join(',')); // 20,6
```

§6.2 对象与原型链:JS 的核心机制

💡 直觉先行:属性找不到,就沿着链向上找

每个 JS 对象都藏着一个"隐藏指针"指向另一个对象——它的**原型(prototype)**。读属性时,如果对象自己没有,引擎就去原型上找,再没有就继续向上,直到 `Object.prototype`,再没有返回 `undefined`。这条查找链就叫**原型链**。所有"继承来的方法"都是这样被找到的: `[1,2].push` 就是沿着数组 $\rightarrow$ `Array.prototype` $\rightarrow$ `Object.prototype` 找到的。

```
// 三个主角:prototype、__proto__、Object.create
// 1. __proto__:对象的隐藏原型指针(现在推荐用 Object.getPrototypeOf 读)
const dog = { bark() { return '汪汪'; } };
const puppy = { name: '小黄' };
Object.setPrototypeOf(puppy, dog); // 设置 puppy 的原型为 dog
console.log(puppy.bark());           // 汪汪 ← 自己没有,从原型 dog 找到
console.log(puppy.name);             // 小黄 ← 自己的属性
console.log(Object.getPrototypeOf(puppy) === dog); // true

// 2. Object.create(proto):以 proto 为原型创建新对象(最干净的写法)
const base = { greet() { return '你好'; } };
const child = Object.create(base);
child.age = 3;
console.log(child.greet());          // 你好 ← 原型链查找
console.log(child.age);              // 3 ← 自身属性

// 3. 原型链是一串:child → base → Object.prototype → null
console.log(child.greet === base.greet); // true,是同一个函数
console.log(Object.getPrototypeOf(Object.prototype)); // null,链的终点

// 4. 属性查找 vs 属性写入:读沿链走,写只落自身
const parent = { count: 1 };
const kid = Object.create(parent);
kid.count = 100;                     // 写自己的属性,不动原型
console.log(kid.count, parent.count); // 100 1
console.log(kid.hasOwnProperty('count')); // true,确认是自己的

// 5. hasOwnProperty:区分"自己的"与"继承的"
console.log(kid.hasOwnProperty('greet')); // false ← greet 来自原型
console.log('greet' in kid);              // true ← in 会沿链查找
```

⚠ 常见误区 6.1:直接改 __proto__ 是危险的

`__proto__` 是历史遗留的访问器,性能差且可读性糟;生产代码用 `Object.create` 建链、`Object.getPrototypeOf/setPrototypeOf` 查改。另外对象字面量不能直接写 `__proto__` 键以外的原型: `{ __proto__: base }` 是合法的设置原型语法(字面量特例),但团队规范通常统一用 `Object.create` 更清晰。

§6.3 构造函数与 new 的过程

ES5 时代的"类"就是构造函数 + 原型方法的组合。new 一个函数时,引擎做四件事:① 创建空对象;② 把空对象的原型指向构造函数的 prototype;③ 以该对象为 this 执行构造函数体;④ 返回该对象(除非构造函数显式返回对象)。

```
// 构造函数:约定首字母大写(与 C 的命名规范类似)
function Animal(name) {
  this.name = name;           // ③ this 是新建的空对象
}
// 方法挂到 prototype:所有实例共享,不重复创建
Animal.prototype.speak = function () {
  return `${this.name} 叫了一声`;
};
Animal.prototype.kind = '动物';

const a1 = new Animal('小狗'); // 完整四步
const a2 = new Animal('小猫');
console.log(a1.speak());       // 小狗 叫了一声
console.log(a1.speak === a2.speak); // true ← 共享同一个函数
console.log(a1.kind);          // 动物 ← 从原型拿的
console.log(a1 instanceof Animal); // true

// 手动实现 new 的等价过程(理解原理):
function myNew(Ctor, ...args) {
  const obj = Object.create(Ctor.prototype); // ① + ②
  const ret = Ctor.apply(obj, args);         // ③
  return (ret !== null && typeof ret === 'object') ? ret : obj; // ④
}
const a3 = myNew(Animal, '小兔');
console.log(a3.speak());        // 小兔 叫了一声
```

⚠ 常见误区 6.2:忘了 new 直接调用构造函数

`Animal('狗')` 不报错,但 `this` 走默认绑定——非严格模式会创建全局变量 `name`,严格模式抛 `TypeError`。生产防范:① 构造函数内第一行 `if (!new.target) throw new Error('必须用 new 调用');` ② 或直接用 `class`(`class` 构造函数忘了 `new` 一定抛错,这是 `class` 优于函数构造的安全点)。

§6.4 class 语法糖:背后的原型

ES2015 的 `class` 不引入新机制,它只是把"构造函数 + prototype + new"包成更清晰的语法。下面的 `class` 与上面的 `Animal` 是等价的:

```

class Animal {
  // 构造函数
  constructor(name) {
    this.name = name;
  }
  // 原型方法:挂在 Animal.prototype 上
  speak() {
    return `${this.name} 叫了一声`;
  }
  // 静态方法:挂在 Animal 本身上($6.7)
  static create(name) {
    return new Animal(name);
  }
}

const a = Animal.create('小牛');
console.log(a.speak());           // 小牛 叫了一声
console.log(a instanceof Animal); // true

// 验证"语法糖":方法确实在 prototype 上
console.log(typeof Animal);       // 'function' ← class 本质是函数
console.log(a.speak === Animal.prototype.speak); // true
console.log(Object.getOwnPropertyNames(Animal.prototype)); // [ 'constructor', 'speak' ]

// class 与函数构造的差异(class 更严格):
// 1. 必须 new 调用
// 2. 不能作为普通函数调用
// 3. 方法不可枚举、不可被 for...in 看到
for (const k in a) {
  console.log('可枚举:', k);      // 只有 name,方法不可枚举
}

```

6.4.1 继承:extends 与 super

```
class Dog extends Animal {
  constructor(name, breed) {
    super(name);          // 必须先调用 super 再访问 this
    this.breed = breed;
  }
  // 重写方法:覆盖父类实现
  speak() {
    return `${super.speak()}!(${this.breed})`; // super.speak 调父类版
  }
  fetch() {
    return `${this.name} 叼回飞盘`;
  }
}

const d = new Dog('旺财', '金毛');
console.log(d.speak());          // 旺财 叫了一声!(金毛)
console.log(d.fetch());          // 旺财 叼回飞盘
console.log(d instanceof Dog);    // true
console.log(d instanceof Animal); // true ← 原型链:实例→Dog.prototype→Animal.prototype
console.log(Object.getPrototypeOf(Dog.prototype) === Animal.prototype); // true

// 继承的本质就是原型链延长:
// d → Dog.prototype → Animal.prototype → Object.prototype → null
// 查找 speak:先看 d,再看 Dog.prototype(命中,重写版)
```

⚠ 常见误区 6.3:派生类构造器忘了 super

派生类(有 extends)的 constructor 里,在访问 this 之前必须先调用 super(...)——否则抛 ReferenceError。原因:this 由父类构造器初始化(与 C++ 的"基类先构造"同理)。若子类不写 constructor,引擎自动补 constructor(...args) { super(...args); }。

6.4.2 手写原型式继承:老代码能读懂

```
// 旧式继承:Object.create 手动连链(ES5 时代标准写法)
function OldDog(name, breed) {
  Animal.call(this, name); // 借用父构造函数初始化 this
  this.breed = breed;
}
OldDog.prototype = Object.create(Animal.prototype); // 链:OldDog.prototype 的原型是 Animal.prototype
OldDog.prototype.constructor = OldDog;             // 修正 constructor 指向
OldDog.prototype.speak = function () {
  return Animal.prototype.speak.call(this) + '!(旧式继承)';
};
const old = new OldDog('阿黄', '土狗');
console.log(old.speak()); // 阿黄 叫了一声!(旧式继承)
console.log(old instanceof OldDog); // true
```

§6.5 访问控制:# 私有字段

JS 没有 public/protected/private 修饰符。C++ 的 private 对应物是 # 私有字段(ES2022):真正的语言级私有,类外无法访问,包括 Object.keys、反射都看不到。访问控制只有两档:public(默认)与 #private;protected 没有直接对应,惯用法是用 # 私有 + 受保护方法放原型(文档约定)。

```
class BankAccount {
  #balance;           // 私有字段:必须先在类里声明
  #owner;

  constructor(owner, initial) {
    this.#owner = owner;
    this.#balance = initial;
  }

  deposit(n) {
    if (n <= 0) throw new Error('存款必须为正');
    this.#balance += n;
  }

  withdraw(n) {
    if (n > this.#balance) return false;
    this.#balance -= n;
    return true;
  }

  get balance() {      // 只读出口(getter, §6.6)
    return this.#balance;
  }
}

const acc = new BankAccount('李雷', 100);
acc.deposit(50);
console.log(acc.balance);      // 150
console.log(acc.withdraw(200)); // false ← 余额不足
console.log(acc.balance);      // 150
// console.log(acc.#balance); // ✗ SyntaxError:私有字段类外不可见!
// console.log(acc['#balance']); // ✗ 也不行,私有字段不是普通键

// 私有方法:#method() 同样类外不可调
class Counter {
  #count = 0;
  #valid(n) { return n >= 0; }
  add(n) {
    if (!this.#valid(n)) throw new Error('不能加负数');
    this.#count += n;
  }
  get value() { return this.#count; }
}

const c = new Counter();
c.add(3);
console.log(c.value);          // 3
```


工程建议 6.1:私有字段 vs 闭包 vs 下划线约定

三种"私有"方案对比:① `#private` 语言级强制,ES2022+,生产首选;② 闭包私有(第4章账户例子)灵活但每个实例一份闭包内存,且无法被 `this` 调用;③ `_prefix` 只是命名约定,不防君子不防小人,仅用于"文档化意图"。写库(给别人用)必须用 `#`;写应用内部代码,团队约定一致即可,但 ESLint 的 `no-underscore-dangle` 常强制用 `#`。

§6.6 getter / setter:访问器属性

C 的结构体字段是裸数据;生产代码常用 `getter/setter` 控制读写(校验、计算属性、惰性求值)。语法:`get/set` 定义在类里,或用 `Object.defineProperty` 定义在对象上。

```
class Temperature {
  #celsius;
  constructor(c) { this.#celsius = c; }

  // 读:celsius → 华氏
  get fahrenheit() {
    return this.#celsius * 9 / 5 + 32;
  }
  // 写:从华氏反推摄氏(带校验)
  set fahrenheit(f) {
    if (!Number.isFinite(f)) throw new Error('非法温度');
    this.#celsius = (f - 32) * 5 / 9;
  }
  get celsius() { return this.#celsius; }
}
const t = new Temperature(100);
console.log(t.fahrenheit);    // 212
t.fahrenheit = 32;
console.log(t.celsius);       // 0 ← setter 反向换算

// 惰性求值:第一次访问才计算
class Expensive {
  #cache = null;
  get data() {
    if (this.#cache === null) {
      console.log('首次计算...');
      this.#cache = { heavy: true, at: Date.now() };
    }
    return this.#cache;
  }
}
const e = new Expensive();
console.log(e.data.at > 0);    // 首次计算... true
console.log(e.data === e.data); // true,第二次直接命中缓存
```

§6.7 静态成员与 instanceof

```
class MathUtils {
  static PI = 3.14159265358979;           // 静态字段(ES2022)
  static square(n) { return n * n; }      // 静态方法
  #secret = '实例私有';

  static create() { return new MathUtils(); }
}
// 静态成员挂在类本身,不挂在实例
console.log(MathUtils.PI);                // 3.14159265358979
console.log(MathUtils.square(9));          // 81
const mu = MathUtils.create();
// console.log(mu.PI);                    // undefined ← 实例没有静态成员
// console.log(mu.square);                // undefined

// 静态方法里的 this 是类本身(用于工厂/单例)
class Config {
  static #instance = null;
  static get() {
    if (this.#instance === null) {
      this.#instance = new Config();
    }
    return this.#instance;                // 单例模式
  }
}
console.log(Config.get() === Config.get()); // true,同一个实例

// instanceof:沿着原型链查
console.log([] instanceof Array);          // true
console.log([] instanceof Object);         // true ← 链更长也能匹配
console.log(mu instanceof MathUtils);       // true
// 跨 realm 陷阱:iframe/worker 里的数组 instanceof 主窗口 Array 为 false
console.log(Array.isArray([]));             // true ← 生产用 Array.isArray 更稳
```

§6.8 多态与鸭子类型

C 的多态靠函数指针表 + 强制转换;JS 的多态天然:只要对象有同名方法,就能被同样调用——鸭子类型("走起来像鸭子、叫起来像鸭子,它就是鸭子")。不需要接口声明,不需要继承关系。

```
// 鸭子类型:三个无关类,有相同方法即可被统一调用
class Dog {
  constructor(name) { this.name = name; }
  speak() { return `${this.name}:汪汪`; }
}
class Cat {
  constructor(name) { this.name = name; }
  speak() { return `${this.name}:喵喵`; }
}
class Robot {
  constructor(name) { this.name = name; }
  speak() { return `${this.name}:哔哔(检测到人类)`; }
}

// 多态调用:不关心具体类型
function callAll(animals) {
  return animals.map((a) => a.speak()); // 只要求有 speak 方法
}
console.log(callAll([new Dog('大黄'), new Cat('小橘'), new Robot('R2')]));
// [ '大黄:汪汪', '小橘:喵喵', 'R2:哔哔(检测到人类)' ]

// 生产防御:先验证方法存在(可选链 + typeof)
function safeSpeak(any) {
  if (typeof any?.speak !== 'function') {
    return '无法说话';
  }
  return any.speak();
}
console.log(safeSpeak(new Dog('小黑'))); // 小黑:汪汪
console.log(safeSpeak({})); // 无法说话
```

★ 重点结论:接口在 JS 里是"约定"不是"类型"

C++/Java 用接口/抽象类在 编译期 保证"一定实现某方法";JS 在 运行期 靠鸭子类型,接口只是文档(或用 JSDoc/TypeScript 标注,见第14章)。代价是错误后置:少写了 speak 方法,运行时才报错。生产缓解:入口处校验(如上面的 safeSpeak)、TypeScript、以及充分的单元测试。

§6.9 设计原则:组合优于继承

原型继承是单链,深了会又脆又难懂(经典反例:Animal→Mammal→Dog→Poodle 五层链)。JS 社区共识:多用组合(对象持有对象),少用继承(对象是另一个对象的子类)。判断标准:有"is-a"关系(狗是动物)且复用方法时用继承;只是"需要某能力"时用组合。

```
// 组合示例:飞行/游泳能力用对象拼装,而不是造"会飞的狗"类
const flyable = (name) => ({
  fly() { return `${name} 飞起来了`; },
});
const swimmable = (name) => ({
  swim() { return `${name} 游了起来`; },
});

class Duck {
  constructor(name) {
    this.name = name;
    Object.assign(this, flyable(name), swimmable(name)); // 组合能力
  }
}

const d2 = new Duck('唐老鸭');
console.log(d2.fly());           // 唐老鸭 飞起来了
console.log(d2.swim());          // 唐老鸭 游了起来

// 对照:继承路线要造 FlyAnimal→SwimAnimal→FlyingSwimmingDuck,层级爆炸

// 工厂 + 组合:完全不用 class 的"对象工厂"风格同样优秀
function createLogger(level = 'info') {
  const timestamps = [];
  return {
    log(msg) {
      timestamps.push(Date.now()); // 闭包私有状态
      console.log(`[${level}] ${msg}`);
    },
    count() { return timestamps.length; },
  };
}

const logger = createLogger('warn');
logger.log('磁盘空间不足');
logger.log('连接超时');
console.log('日志条数:', logger.count()); // 2
```

工程建议 6.2:生产中的类设计检查表

① 类层级不超过 2 层(父 + 子);再深先考虑组合;② 每个类单一职责,方法数超过 15 个考虑拆分;③ 私有状态全部 #,对外只暴露最小方法集;④ 依赖注入:构造器接收依赖对象,不用全局 new,便于测试;⑤ 优先"对象工厂 + 组合"起步,真需要继承时再加 extends——少即是多。

本章速查表

主题	速查要点
对象本质	属性表 + 隐藏原型指针;属性查找沿原型链向上
原型链主角	prototype(构造函数的)、__proto__(对象的,历史)、Object.create(建链)
读属性 vs 写属性	读沿链找;写只落自身属性
hasOwnProperty	区分自己的/继承的;in 运算符沿链查找
new 四步	建空对象 → 连原型 → this 执行 → 返回对象
class	语法糖:本质是函数 + prototype;方法不可枚举
extends/super	链变长;派生类构造器必须先 super 再 this
旧式继承	Object.create(父.prototype) + 修正 constructor
私有	#field/#method 语言级私有;无 protected,用 #+ 文档约定
getter/setter	读写拦截、计算属性、惰性求值
静态成员	static 挂类本身,实例不可见;静态 this 是类
instanceof	沿原型链查;跨 realm 失效;数组用 Array.isArray
多态	鸭子类型:有同名方法即可统一调用,无需接口
组合优先	is-a 用继承;要能力用组合/混入;层级 ≤2

最小必会清单

- 能解释原型链查找过程,并画出 实例→原型→Object.prototype→null 链
- 能用 Object.create 建链并验证 hasOwnProperty 的区别
- 能背诵 new 的四步并手写 myNew
- 能写出 class 与"构造函数 + prototype"的等价转换
- 能说出 extends 后原型链的形态(Dog.prototype 的原型是 Animal.prototype)
- 能写出 # 私有字段的类,并说出类外访问的报错
- 能用 getter/setter 实现"摄氏/华氏双向换算"
- 能解释鸭子类型多态并给出一个统一调用示例
- 能说出"组合优于继承"的判断标准
- 能说出 instanceof 的跨 realm 陷阱与 Array.isArray

自测题

- Q** 1. 写出输出:const p = {a:1}; const c = Object.create(p); console.log(c.a, c.hasOwnProperty('a'), 'a' in c)

✅ 参考答案

1、false、true。c 自己没有 a,查找沿原型链命中 p.a=1;hasOwnProperty 只看自身;in 沿链查找所以 true。

Q 2. class 与构造函数定义"类"的差别有哪些?(至少 3 条)

✅ 参考答案

① class 必须 new 调用,普通调用抛错;② class 方法不可枚举;③ class 语法更清晰,支持 extends/super/static/# 私有;④ class 有块级作用域(不提升);底层都是原型链。

Q 3. 派生类构造器里为什么必须先 super()?不写会怎样?

✅ 参考答案

this 由父类构造器初始化,访问 this 前必须完成 super;违反时抛 ReferenceError。若子类不写构造器,引擎自动补 super(...args)。

Q 4. 写出用 Object.create 手工实现继承(Animal → OldDog)的关键四行,并说明每行作用。

✅ 参考答案

① Animal.call(this, name) 借用父构造初始化;② OldDog.prototype = Object.create(Animal.prototype) 连链;③ OldDog.prototype.constructor = OldDog 修正指向;④ 在 prototype 上覆盖方法。

Q 5. 如何让类外代码读不到 #balance 但能读 balance?这两种方式分别叫什么?

✅ 参考答案

声明 #balance 私有字段,对外提供 get balance() { return this.#balance; }。前者是私有字段(语言级),后者是 getter 访问器。

Q 6. 一个"能飞能游"的对象,组合与继承各怎么写?生产推荐哪个?

✅ 参考答案

组合: Object.assign(this, flyable(), swimmable());继承需要造中间层类,层级爆炸。生产推荐组合。

章末实验题:几何图形系统

实验 6:几何图形系统(继承 + 多态 + 私有 + 访问器)

实验目标:实现圆形/矩形/三角形组成的图形系统:基类定义接口,子类实现面积与周长,支持统一汇总,全面覆盖本章知识点。

实验要求:

- 任务 1(覆盖:class、构造器、super):基类 Shape 存 name;Circle/Rect/Triangle 继承并用 super 初始化
- 任务 2(覆盖:# 私有字段、getter/setter):Circle 用 #radius 私有字段 + get/set radius 校验非负;Rect 用 #w/#h
- 任务 3(覆盖:多态、鸭子类型):统一接口 area()/perimeter() 三个子类各自实现;汇总函数只管调用,不查类型
- 任务 4(覆盖:原型链、instanceof):遍历图形数组输出 instanceof 结果与原型链验证(Dog 式链)
- 任务 5(覆盖:静态成员、工厂):Shape.create(type, ...) 静态工厂按类型造图形;MathUtils 式静态常量 PI
- 任务 6(覆盖:组合优先):实现"可缩放的图形" mixin(scale 方法),用 Object.assign 组合进 Shape,而不是造子类
- 任务 7(覆盖:Object.create、hasOwnProperty):用 Object.create 演示一个"临时图形"对象,验证属性归属

实验环境:Node.js 20+,运行 `node shapes.js`。

第一步 写基类:Shape 定义 constructor(name) 与接口方法(抛"未实现"错误)

第二步 写三个子类:各自实现 area/perimeter;注意先 super 再访问 this

第三步 mixin 与工厂:scalable 混入、Shape.create 工厂、演示与汇总输出

✅ 参考答案要点

核心:基类接口 + 三子类多态 + #私有字段与访问器 + 静态工厂 + mixin 组合。约 130 行,覆盖全章知识点。


```
// shapes.js — Node.js 20+ 运行:node shapes.js
'use strict';

// 1. 基类:定义接口(覆盖:class、构造器、多态契约)
class Shape {
  constructor(name) {
    this.name = name;
  }
  area() {
    throw new Error(`${this.name} 未实现 area()`);
  }
  perimeter() {
    throw new Error(`${this.name} 未实现 perimeter()`);
  }
  describe() {
    return `${this.name}: 面积 ${this.area().toFixed(2)},周长 ${this.perimeter().toFixed(2)}`;
  }
}

// 2. 圆形:# 私有字段 + getter/setter 校验(覆盖:私有、访问器)
class Circle extends Shape {
  #radius;
  constructor(radius) {
    super('圆形');
    this.radius = radius;      // 走 setter 校验
  }
  get radius() { return this.#radius; }
  set radius(r) {
    if (!Number.isFinite(r) || r <= 0) {
      throw new Error('半径必须为正数');
    }
    this.#radius = r;
  }
  area() { return Math.PI * this.#radius ** 2; }
  perimeter() { return 2 * Math.PI * this.#radius; }
}

// 3. 矩形:两个 # 私有字段
class Rect extends Shape {
  #w;
  #h;
  constructor(w, h) {
    super('矩形');
    if (w <= 0 || h <= 0) throw new Error('宽高必须为正');
    this.#w = w;
    this.#h = h;
  }
  area() { return this.#w * this.#h; }
  perimeter() { return 2 * (this.#w + this.#h); }
}

// 4. 三角形:海伦公式
class Triangle extends Shape {
  #a;
  #b;
```

```

#c;
constructor(a, b, c) {
  super('三角形');
  if (a + b <= c || a + c <= b || b + c <= a) {
    throw new Error('三边无法构成三角形');
  }
  this.#a = a; this.#b = b; this.#c = c;
}
area() {
  const s = this.perimeter() / 2;
  return Math.sqrt(s * (s - this.#a) * (s - this.#b) * (s - this.#c));
}
perimeter() { return this.#a + this.#b + this.#c; }
}

```

// 5. 静态工厂 + 静态常量(覆盖:static、单例式出口)

```

class ShapeFactory {
  static PI = Math.PI;
  static create(type, ...args) {
    switch (type) {
      case 'circle': return new Circle(...args);
      case 'rect': return new Rect(...args);
      case 'triangle': return new Triangle(...args);
      default: throw new Error(`未知图形类型:${type}`);
    }
  }
}

```

// 6. 组合:mixin 混入缩放能力(覆盖:组合优于继承)

```

const scalable = (self) => ({
  scale(factor) {
    if (factor <= 0) throw new Error('缩放系数必须为正');
    // 通过实例的 radius/w/h 访问器缩放(鸭子类型)
    if (typeof self.radius === 'number') {
      self.radius *= factor;
    } else if (self instanceof Rect) {
      // 矩形私有字段只能通过内部方法缩放,这里演示"组合兜底"
      self._scaleInternal?.(factor);
    }
    return self;
  },
});

```

// 为 Rect 提供内部缩放钩子(私有字段 + 内部方法)

```

Rect.prototype._scaleInternal = function (f) {
  this.#w *= f;
  this.#h *= f;
};

```

// 7. 鸭子类型多态汇总(覆盖:多态、instanceof 验证)

```

function summarize(shapes) {
  let total = 0;
  for (const s of shapes) {
    console.log(`- ${s.describe()}`);
    console.log(`  instanceof Shape: ${s instanceof Shape}`);
  }
}

```

```

    total += s.area();
  }
  return total;
}

// 8. Object.create 演示(覆盖:Object.create、hasOwnProperty)
function protoDemo() {
  const proto = { tag: '几何对象', log() { console.log('来自原型的方法'); } };
  const tmp = Object.create(proto);
  tmp.name = '临时图形';
  console.log('tmp.tag(来自原型):', tmp.tag);
  console.log('hasOwnProperty:', tmp.hasOwnProperty('tag'), tmp.hasOwnProperty('name'));
  console.log('in 运算符:', 'tag' in tmp);
}

// 9. 主流程
function main() {
  // 工厂创建 + 组合缩放
  const shapes = [
    ShapeFactory.create('circle', 5),
    ShapeFactory.create('rect', 4, 6),
    ShapeFactory.create('triangle', 3, 4, 5),
  ];
  Object.assign(shapes[0], scalable(shapes[0])); // 给圆形混入缩放
  shapes[0].scale(2); // 半径 5 → 10

  const total = summarize(shapes);
  console.log('总面积:', total.toFixed(2));

  // 访问器校验演示
  try {
    const bad = new Circle(-1);
    console.log(bad.describe());
  } catch (err) {
    console.log('非法半径被拦截:', err.message);
  }

  // 私有字段不可类外访问(注释掉的代码会报语法错)
  // console.log(shapes[0].#radius); // ✖ SyntaxError

  // 原型链验证
  const c = shapes[0];
  console.log('原型链:', c instanceof Circle, c instanceof Shape, c instanceof Object);
  console.log('Circle.prototype 的原型是 Shape.prototype:',
    Object.getPrototypeOf(Circle.prototype) === Shape.prototype);

  protoDemo();
}

main();

```

设计说明:Shape 用"抛锚"定义接口契约(鸭子类型语言中接口的常见写法);Circle 的 setter 拦截非法半径;scalable mixin 演示组合优于继承——能力用 Object.assign 混入而不是造"可缩放圆"子类;summarize 只依赖

area()/describe() 接口,不看具体类型;静态工厂集中创建逻辑;protoDemo 收尾验证 Object.create 与原型的属性归属。运行输出包含三种图形的面积周长、总面积、非法输入拦截、原型链验证与原型演示五部分。

下一章预告:第7章 文件I/O —— fs 模块的读写、路径与目录、流式大文件处理、JSON 序列化,与 C 的 fopen/fread 全家对照。